

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

CORSO DI PROGRAMMAZIONE DISPOSITIVI MOBILI 2025/2026

UNIMIBCHESS

Documentazione di Progetto

Gruppo: Gang Del Software

Magdi Fekri Israil Saad Fadi	926207
Lizartigues Clarence	886281
Villostas Mike Zen Stefano	919844
Pena Andrea	887462



Anno Accademico 2025/2026

Indice

1	Introduzione a UNIMIBCHESS	2
2	Principali tecnologie utilizzate	2
3	Architettura dell'applicazione	2
3.1	Struttura	2
3.2	Generazione dei puzzle dal database Lichess	4
3.3	Modalità ospite e utilizzo offline	4
3.4	Elementi di UI	5
3.5	API	5
3.6	Activities	5
3.6.1	Login Activity	5
3.6.2	Home Activity	6
3.6.3	Game Activity	7
3.7	Fragments	9
3.7.1	Login Fragment	9
3.7.2	Register Fragment	9
3.7.3	Reset Password Fragment	9
3.7.4	Profile Fragment	10
3.7.5	Quiz Fragment	10
3.7.6	Play Fragment	10
3.7.7	Settings Fragment	10
4	Librerie utilizzate	11
5	Sviluppi futuri	11

1 Introduzione a UNIMIBCHESS

UNIMIBCHESS è un'applicazione pensata per gli appassionati di scacchi che vogliono migliorare le proprie abilità tattiche attraverso la risoluzione di puzzle scacchistici. L'app propone una vasta gamma di esercizi basati su situazioni reali di gioco, ricavati dal database pubblico di puzzle di *Lichess*. Ogni livello sfida l'utente a trovare la mossa migliore o a risolvere una combinazione vincente, aiutando così a sviluppare la visione strategica, il calcolo e la capacità decisionale. L'interfaccia è intuitiva e interattiva, rendendo l'esperienza di apprendimento coinvolgente e stimolante. L'applicazione è inoltre utilizzabile anche senza registrazione, grazie alla modalità *ospite* che consente di giocare offline.

2 Principali tecnologie utilizzate

L'applicazione è stata sviluppata in ambiente Android nativo seguendo il pattern architetturale MVVM (Model–View–ViewModel). Le principali tecnologie e librerie impiegate sono le seguenti:

- **Java** come linguaggio di sviluppo principale.
- **Android SDK** e i componenti di *Android Jetpack* (ViewModel, LiveData, Navigation).
- **Room** per la persistenza dei dati su database locale.
- **Firestore** (Authentication e Realtime Database) per l'autenticazione e il salvataggio dei dati utente.
- **Retrofit** per la comunicazione di rete con i servizi REST esterni.
- **Stockfish** come motore scacchistico per il calcolo delle mosse ottimali.
- **Lichess Puzzle Database** come fonte dei puzzle tattici: i livelli della modalità quiz vengono generati a partire dal database aperto di puzzle di Lichess, distribuito in formato CSV e incluso negli asset dell'applicazione.

La Figura 1 sintetizza l'architettura MVVM adottata e il flusso dei dati tra i vari strati, dall'interfaccia utente fino alle fonti di dati locali (Room) e remote (Retrofit/Firebase).

3 Architettura dell'applicazione

3.1 Struttura

L'applicazione è composta dai seguenti strati:

- **Activity e Fragment:** questi componenti gestiscono l'interfaccia utente (UI) dell'applicazione, popolando le View e gestendo le transizioni tra le varie schermate. L'acquisizione dei dati è mediata dai *LiveData*, che sono integrati nei ViewModel.
- **ViewModel:** funziona come uno strato di mediazione tra la UI e i livelli di dati, specificatamente i repository. Il ViewModel fornisce alla UI i riferimenti ai LiveData, che a loro volta ricevono i dati dai Repository. Una volta che i LiveData sono aggiornati, la UI viene notificata e può leggere i nuovi dati per aggiornare le View. Un grande vantaggio dell'utilizzo dei ViewModel è che questi persistono indipendentemente dal ciclo di vita (lifecycle) dei Fragment. I ViewModel hanno inoltre la responsabilità della logica di gioco.

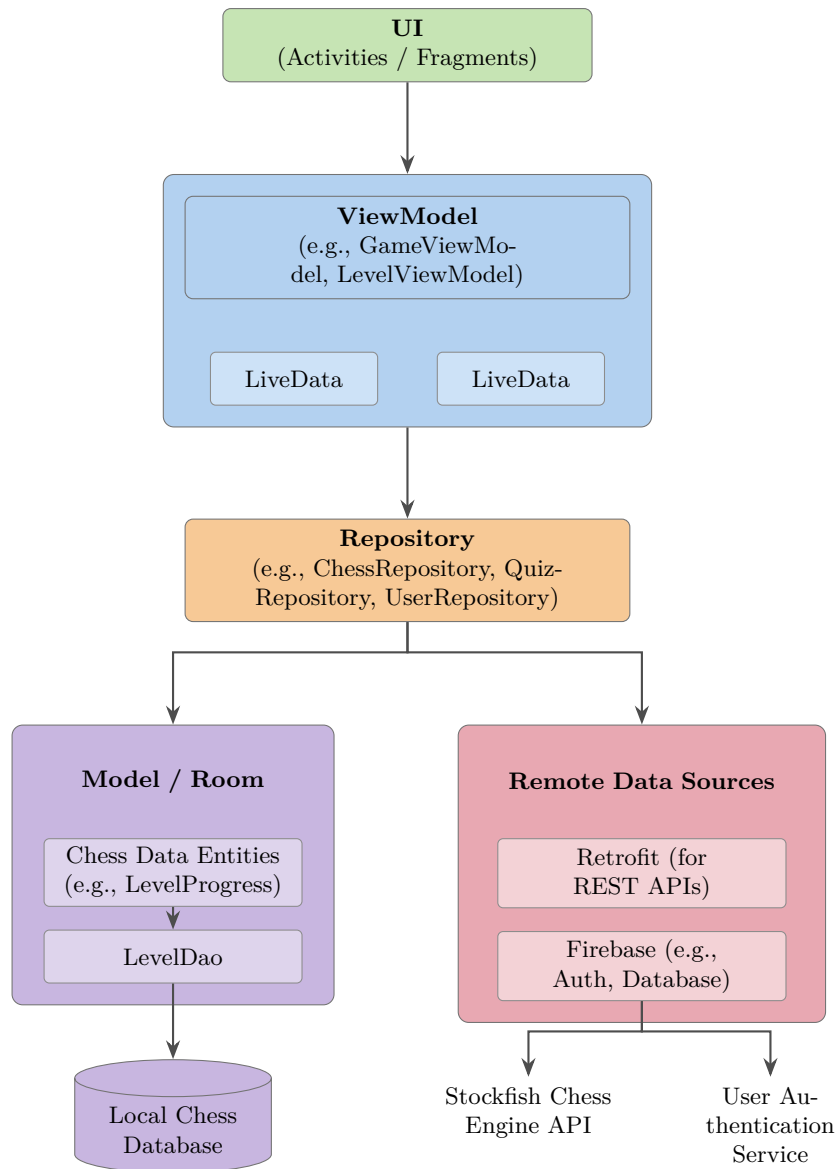


Figura 1: Architettura MVVM di UNIMIBCHESS e flusso dei dati tra gli strati.

- **Repository:** questo componente astrae le fonti di dati e offre un'interfaccia di alto livello per l'accesso ai dati provenienti da database e API esterne, attraverso l'uso di classi DataSource. Il Repository ha anche il compito di notificare ai ViewModel gli aggiornamenti dei dati, che vengono poi riflessi nei LiveData. Per favorire la testabilità, i repository non dipendono direttamente dal Context Android ma da interfacce iniettate (*dependency injection*), come `IResourceProvider` per l'accesso alle risorse stringa e `IQuizDataSource` per il flusso dei dati dei puzzle; in questo modo è possibile testarli con implementazioni fittizie in puri unit test JVM.

Servizi e fonti dati esterne utilizzate:

- **API esterne:** comunicazione con il motore scacchistico Stockfish per il calcolo delle mosse.
- **Room:** gestione di un database locale sul dispositivo.
- **Lichess Puzzle Database:** database CSV di puzzle tattici incluso negli asset, da cui vengono generati i livelli della modalità quiz (si veda la Sezione 3.2).

- **Firestore:**

- *Authentication*: si occupa del login tramite email e password oppure tramite account Google.
- *Realtime Database*: si occupa del salvataggio dell'utente e delle eventuali statistiche.

3.2 Generazione dei puzzle dal database Lichess

Una delle principali evoluzioni dell'applicazione riguarda la provenienza dei livelli della modalità quiz: anziché essere definiti manualmente nel codice, i puzzle vengono ora **prelevati dal database aperto di Lichess**, una raccolta di milioni di posizioni tattiche ricavate da partite reali. Un estratto del database, in formato CSV, è incluso negli asset dell'applicazione: in questo modo il caricamento dei livelli non richiede alcuna connessione di rete ed è quindi disponibile anche offline.

Il flusso di caricamento è gestito da due componenti principali:

- **LichessPuzzleLoader**: si occupa del parsing del CSV. Per ogni riga del database legge la posizione iniziale in notazione *FEN* (Forsyth–Edwards Notation), la sequenza di mosse della soluzione in notazione *UCI*, il *rating* di difficoltà e i *temi tattici* associati (ad esempio `mateIn1`, `backRankMate`, `smotheredMate`). La FEN viene convertita nella matrice di pezzi utilizzata dal motore di gioco interno e la prima mossa UCI (quella dell'avversario) viene applicata automaticamente alla scacchiera, così che il puzzle si presenti all'utente già dal punto di vista del giocatore che deve trovare la soluzione.
- **QuizRepository**: orchestra il caricamento ricevendo il flusso del CSV da un `IQuizDataSource` e applicando i filtri di selezione (intervallo di rating, tema tattico, numero massimo di livelli). I titoli dei livelli vengono generati dinamicamente in base al tema tattico del puzzle tramite `IResourceProvider`, garantendo la localizzazione delle stringhe.

Il caricamento è inoltre **robusto rispetto ai dati corrotti**: le righe malformate del CSV (campi mancanti, rating non numerici, FEN non valide) vengono ignorate senza interrompere il caricamento, e i puzzle non supportati dal motore interno (ad esempio quelli contenenti promozioni o arroccchi nella soluzione) vengono scartati. Qualora il caricamento dal CSV non producesse alcun livello (file assente, vuoto o illeggibile), il repository fornisce dei **livelli di fallback** definiti internamente, così da garantire in ogni caso un'esperienza di gioco funzionante.

3.3 Modalità ospite e utilizzo offline

L'applicazione è ora utilizzabile anche **senza registrazione e senza connessione a Internet**. Dalla schermata di login è infatti possibile accedere come *ospite* (*guest*): in questa modalità l'utente salta completamente l'autenticazione Firebase e accede direttamente alle funzionalità di gioco.

Le caratteristiche della modalità ospite sono le seguenti:

- **Nessuna dipendenza dalla rete**: i puzzle della modalità quiz provengono dal CSV di Lichess incluso negli asset (Sezione 3.2) e la partita libera in locale ("Gioca con un amico") non richiede servizi esterni, rendendo l'app pienamente fruibile offline.
- **Persistenza locale**: i progressi dell'ospite (livelli completati, impostazioni) vengono salvati esclusivamente sul dispositivo tramite Room, senza alcuna sincronizzazione con il Realtime Database di Firebase.
- **Funzionalità limitate**: le funzioni che richiedono necessariamente la rete o un account, come la sfida contro il bot Stockfish (API remota) e la sincronizzazione delle statistiche sul cloud, non sono disponibili in modalità ospite offline.

3.4 Elementi di UI

- **BottomNavigationView**: utilizzato per navigare tra le principali sezioni dell'app attraverso tab posizionate nella parte inferiore dell'interfaccia.
- **CardView**: utilizzato per la divisione visiva dei settings e per presentare i livelli della modalità quiz sotto forma di schede.
- **TextInputEditText**: campo per l'inserimento di testo. Utilizzato principalmente per l'input di dati come email e password nelle schermate di login e registrazione.
- **Button**: per la gestione dei pulsanti di login, registrazione e accesso come ospite, delle opzioni di gioco e delle interazioni nei menu.
- **TextView**: utilizzate per mostrare testi informativi come lo stato della partita (es. "Scacco"), i timer dei giocatori e i nomi dei livelli.
- **ImageView**: utilizzato all'interno della scacchiera per renderizzare la grafica vettoriale dei singoli pezzi (pedoni, torri, re, regina, ...).
- **SquareGridView (Custom View)**: componente personalizzato utilizzato per la matrice di gioco; garantisce che la scacchiera mantenga proporzioni perfettamente quadrate su qualsiasi dispositivo.

3.5 API

- **Retrofit**: libreria client HTTP utilizzata per mappare le interfacce Java in chiamate di rete REST, facilitando la comunicazione e l'invio di richieste asincrone verso i servizi esterni.
- **StockfishService**: servizio dedicato all'interfacciamento con l'API del motore scacchistico Stockfish, responsabile di inviare la configurazione della scacchiera e ricevere la mossa ottimale calcolata dall'intelligenza artificiale.

3.6 Activities

3.6.1 Login Activity

La Login Activity funge da contenitore per i frammenti associati alla gestione dell'accesso degli utenti; include le funzioni di login, registrazione, reset della password e accesso come ospite.

Funzionalità — **Gestione Fragment**: organizza e gestisce i cambiamenti tra i vari frammenti di Login, Register e Reset Password. **Accesso ospite**: consente di saltare l'autenticazione e accedere direttamente alla Home Activity in modalità guest (Sezione 3.3), utile anche in assenza di connessione.

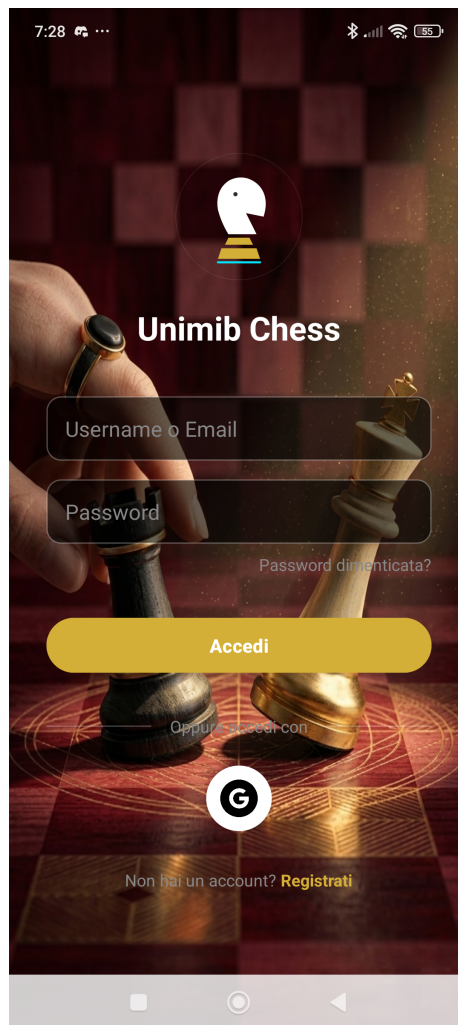


Figura 2: Schermata di Login di UNIMIBCHESS: campi email/password, accesso con Google, link alla registrazione e accesso come ospite.

3.6.2 Home Activity

La Home Activity rappresenta la schermata principale dell'applicazione dopo l'autenticazione (o dopo l'accesso come ospite). Funge da contenitore per i fragment relativi al profilo, alla modalità quiz, alla modalità di gioco e alle impostazioni, gestendo la navigazione tramite la BottomNavigationView.

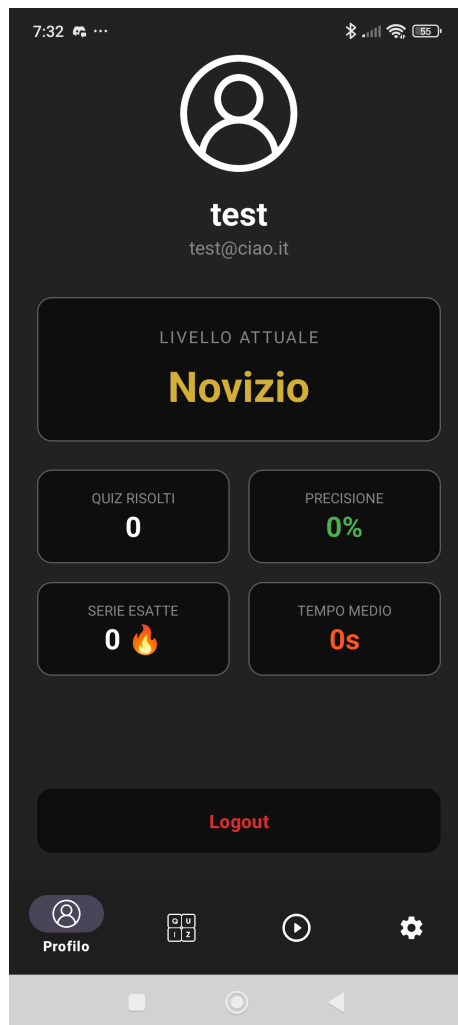


Figura 3: Home Activity con il Profile Fragment attivo: statistiche dell'utente e BottomNavigationView per la navigazione tra le sezioni.

3.6.3 Game Activity

La Game Activity gestisce la schermata di gioco vera e propria, ospitando la scacchiera interattiva (SquareGridView), i timer e le informazioni di stato della partita. L'utente ha a disposizione 30 secondi, per ogni mossa sbagliata viene sottratta 1 vita fino ad un massimo di 3 e nel caso di questo avvenimento l'utente torna nella selezione dei vari livelli Coordina l'interazione tra l'utente e il motore scacchistico Stockfish per il calcolo delle mosse.

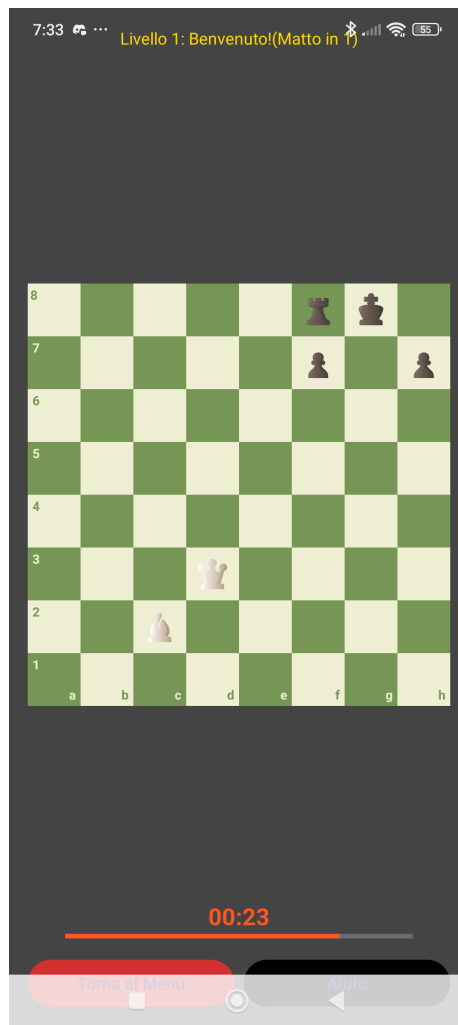


Figura 4: Game Activity durante la risoluzione di un puzzle: scacchiera interattiva (Square-GridView), timer e pulsanti di navigazione e aiuto.

Il giocatore può inoltre decidere se giocare in locale con un amico oppure sfidare il computer: dalla schermata di avvio della partita libera è infatti possibile scegliere tra la modalità “Gioca con un amico”, in cui due utenti si alternano sullo stesso dispositivo, e la modalità “Gioca con bot”, in cui l’avversario è il motore scacchistico Stockfish.

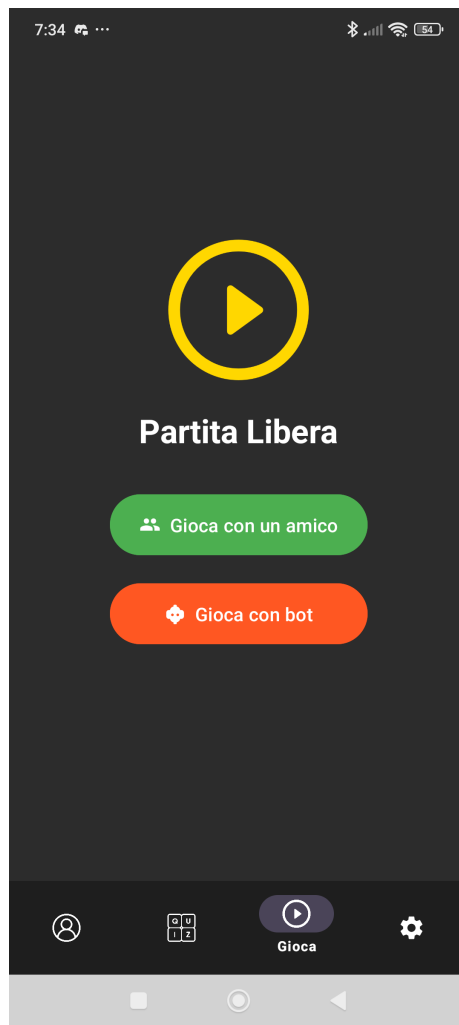


Figura 5: Schermata di selezione della Partita Libera: scelta tra partita locale con un amico o contro il bot.

3.7 Fragments

3.7.1 Login Fragment

Gestisce l'interfaccia di accesso, raccogliendo email e password dell'utente (oppure con Google) e avviando il processo di autenticazione tramite Firebase Authentication. Offre inoltre la possibilità di entrare nell'applicazione come ospite, senza creare alcun account: in questo caso l'autenticazione Firebase viene bypassata e l'app funziona in modalità offline con persistenza locale.

3.7.2 Register Fragment

Permette la creazione di un nuovo account, validando i dati inseriti dall'utente e registrandolo tramite Firebase Authentication.

3.7.3 Reset Password Fragment

Consente all'utente di richiedere il reset della propria password tramite l'invio di un'email di recupero gestita da Firebase.

3.7.4 Profile Fragment

Mostra le informazioni dell'utente e le relative statistiche di gioco, recuperate dal Realtime Database di Firebase. In modalità ospite vengono mostrati i soli dati salvati localmente.

3.7.5 Quiz Fragment

Presenta i livelli della modalità quiz sotto forma di schede (CardView), permettendo all'utente di selezionare e avviare i puzzle scacchistici. I livelli mostrati sono generati dinamicamente a partire dal database di puzzle di Lichess (Sezione 3.2), con titoli che riflettono il tema tattico di ciascun puzzle (matto del corridoio, matto arabo, sacrificio, attacco di scoperta, ...).

3.7.6 Play Fragment

Gestisce l'avvio della modalità di gioco, consentendo all'utente di iniziare una partita contro il motore scacchistico.

3.7.7 Settings Fragment

Permette all'utente di configurare le impostazioni dell'applicazione, organizzate visivamente tramite CardView.

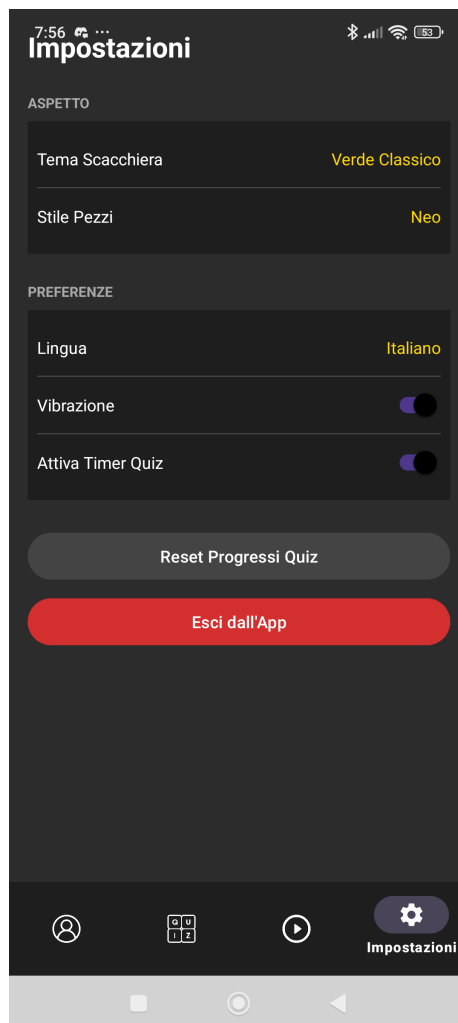


Figura 6: Settings Fragment: impostazioni di aspetto (tema scacchiera, stile pezzi) e preferenze (lingua, vibrazione, timer quiz) organizzate tramite CardView.

4 Librerie utilizzate

- **AndroidX / Jetpack:** ViewModel, LiveData, Navigation, Room.
- **Retrofit e OkHttp:** comunicazione di rete REST.
- **Firebase Authentication e Firebase Realtime Database.**
- **Material Components:** componenti grafici (BottomNavigationView, CardView, TextInputEditText).
- **Stockfish:** motore scacchistico per il calcolo delle mosse.
- **Lichess Puzzle Database:** database CSV open source di puzzle tattici, integrato negli asset dell'applicazione.

5 Sviluppi futuri

Tra i possibili sviluppi futuri dell'applicazione si individuano:

- Introduzione di una modalità multiplayer online tra utenti.
- Ampliamento del database di puzzle con nuove categorie tattiche e filtri di difficoltà selezionabili dall'utente.
- Migrazione dei progressi della modalità ospite verso un account registrato, con sincronizzazione cloud dei dati locali.
- Sistema di classifiche e achievement per aumentare il coinvolgimento.
- Supporto a ulteriori metodi di autenticazione e sincronizzazione cloud.
- Ottimizzazione delle prestazioni del motore scacchistico su dispositivi di fascia bassa.